

SAST para detecção de vulnerabilidades em workflows do n8n

João Pedro Schmidt Cordeiro Gustavo Zambonin

11 de dezembro de 2025

Resumo

A crescente adoção de plataformas Low-Code/No-Code (LCNC), como o n8n, tem transformado o desenvolvimento de automações empresariais, democratizando a criação de workflows através de interfaces visuais. Entretanto, essa mudança de paradigma introduz uma lacuna crítica de governança: enquanto a plataforma subjacente é protegida por práticas robustas de segurança, os workflows criados pelos usuários não são submetidos ao mesmo rigor de análise de segurança. Este trabalho propõe o desenvolvimento de uma ferramenta de Análise Estática de Segurança de Aplicações (SAST) específica para workflows do n8n, capaz de detectar automaticamente vulnerabilidades como SQL Injection, Command Injection, SSRF, exposição de credenciais, manuseio inseguro de dados e negação de serviço. A metodologia baseia-se em uma estratégia híbrida que combina o Agentic Radar (focado em riscos de agentes de IA) e o Semgrep (configurado com regras customizadas para vulnerabilidades tradicionais). Além disso, propõe-se o uso de inteligência artificial para acelerar a criação de regras de detecção. A análise revelou coberturas complementares de 16,7% e 66,7% respectivamente, demonstrando a viabilidade da análise estática de estruturas JSON declarativas. A solução contribui para elevar o nível de segurança das automações desenvolvidas em organizações brasileiras, auxiliando na conformidade com a LGPD.

Palavras-chave: SAST, n8n, Low-Code/No-Code, Segurança de Aplicações, Análise Estática, Vulnerabilidades, Automação de Workflows, Inteligência Artificial, LGPD.

Sumário

1	Introdução	2
1.1	Motivação e Problema	3
1.2	Escopo e Contribuições	3
2	Fundamentação Teórica e Estado da Arte	3
2.1	Ferramentas de SAST para n8n	3
2.2	Explicabilidade e Privacidade	4
2.3	Gaps e Oportunidades	4
3	Metodologia	4
3.1	Análise das Ferramentas	4
3.2	Geração de Regras Semgrep Assistida por IA	5
3.3	Métricas de Validação	5

4	Objetivos e Métricas de Sucesso	5
4.1	Métricas e Benchmarks	6
4.2	Critérios de Sucesso	6
5	Escopo da Solução e Requisitos	6
5.1	Casos de Uso e Exclusões	6
5.2	Arquitetura e Componentes	6
5.3	Stack Tecnológico e Infraestrutura	7
5.4	Dataset de Validação	7
6	Ameaças, Riscos e Controles	7
6.1	Superfícies de Ataque	7
6.2	Análise STRIDE	8
6.3	Riscos de Sistemas com IA	8
6.4	Privacidade (LGPD) e Ética	8
7	Avaliação Experimental	8
7.1	Protocolo Experimental	9
7.1.1	Configuração de Hardware e Software	9
7.1.2	Dataset de Validação	9
7.2	Resultados do Semgrep	9
7.3	Avaliação com Agentic Radar	10
7.4	Análise Comparativa e Abordagem Híbrida	10
7.5	Resultados da Abordagem Híbrida	11
7.5.1	Métricas Agregadas	11
7.5.2	Cobertura por Classe de Vulnerabilidade	11
7.5.3	Validação dos Objetivos	12
7.5.4	Workflow de Integração Proposto	12
7.5.5	Implementação de Referência: workflow_analyzer.py	13
7.5.6	Conclusão da Avaliação Híbrida	15
8	Discussão	16
8.1	Alcance dos Objetivos e Limitações	16
8.2	Comparação com Estado-da-Arte	16
8.3	Trade-offs e Implicações Práticas	16
9	Conclusões e Próximos Passos	17
9.1	Contribuições	17
9.2	Trabalhos Futuros	17
9.3	Recomendações	17
A	Código Completo do Analisador de Workflows	18
B	Regras Semgrep para n8n	26

1 Introdução

A crescente adoção de plataformas Low-Code/No-Code (LCNC) representa uma transformação fundamental no desenvolvimento de automações empresariais. O n8n, plataforma de automa-

ção de workflows de código aberto, exemplifica essa mudança ao permitir que equipes conectem APIs, bancos de dados e serviços através de um editor visual. Esta democratização do desenvolvimento introduz desafios críticos de segurança: o "código-fonte" da aplicação (arquivo JSON do workflow) não está sujeito ao mesmo rigor de segurança que a plataforma na qual é executado.

1.1 Motivação e Problema

O interesse por este tema surge de uma necessidade prática: a plataforma n8n é utilizada diariamente em ambiente de trabalho, e muitos usuários que criam workflows não possuem conhecimento técnico em segurança, resultando no desenvolvimento não intencional de vulnerabilidades. A própria n8n implementa práticas de segurança robustas em seu código-fonte, mas essa varredura não se estende aos workflows criados pelos usuários.

No contexto brasileiro, a LGPD [1] cria demanda específica por ferramentas que garantam conformidade e segurança de automações empresariais. Os workflows podem conter seis classes críticas de vulnerabilidades: **SQL Injection**, **Command/Code Injection**, **Secret Sprawl**, **SSRF**, **Insecure Data Handling** e **Denial of Service**. A natureza declarativa do JSON do n8n paradoxalmente facilita certos tipos de análise estática, declarando explicitamente relações de fluxo de dados através do array `connections`.

1.2 Escopo e Contribuições

Este trabalho foca no desenvolvimento e validação de uma abordagem híbrida para análise estática de segurança de workflows n8n, incluindo: avaliação de Agentic Radar e Semgrep, detecção das seis classes de vulnerabilidades, e metodologia para criação de regras utilizando IA. Estão excluídos: implementação de ferramenta SAST de produção, interface gráfica, análise dinâmica e correção automática de vulnerabilidades.

As principais contribuições são:

- **Análise comparativa** de Agentic Radar e Semgrep com coberturas complementares
- **Validação de viabilidade** da análise estática de estruturas JSON declarativas
- **Abordagem híbrida** combinando ferramentas especializadas e genéricas
- **Metodologia com IA** para acelerar criação de regras de detecção
- **Documentação de lacunas** e oportunidades para pesquisas futuras

2 Fundamentação Teórica e Estado da Arte

Esta seção apresenta a revisão da literatura e análise das ferramentas existentes para análise estática de segurança em plataformas LCNC, focando em duas ferramentas principais: Agentic Radar (solução baseada em agentes de IA) e Semgrep (motor de análise estática extensível). É crucial ressaltar que **não existe uma solução consolidada para detecção de vulnerabilidades especificamente em workflows do n8n**.

2.1 Ferramentas de SAST para n8n

Agentic Radar [12]: Ferramenta de código aberto da SPLX AI para análise de segurança de workflows com agentes de IA. Implementa análise estática consciente do contexto, modo de teste com API OpenAI para testes adversariais [13], e correlação com OWASP LLM Top

10. Foco em riscos de IA: injeção de prompt, vazamento de PII, geração de conteúdo nocivo. Cobertura limitada a aproximadamente 16,7% das classes de vulnerabilidades tradicionais.

Semgrep [11]: Motor de análise estática baseado em correspondência de padrões com AST, suportando 30+ linguagens incluindo JSON [10]. Permite desenvolvimento de regras customizadas em YAML para detectar padrões inseguros específicos. Capacidade de taint tracking para rastrear fluxo de dados contaminados [4]. Cobertura potencial de 66,7% das classes, dependendo do desenvolvimento de regras personalizadas.

Limitação Crítica Identificada: O modo JSON do Semgrep apresenta dificuldades com estruturas aninhadas (4+ níveis) como workflows n8n. A solução foi o **modo genérico**, que trata JSON como texto plano com correspondência via regex — menos sofisticado semanticamente, mas significativamente mais efetivo na prática (Seção 7).

2.2 Explicabilidade e Privacidade

Ambas as ferramentas implementam mecanismos de explicabilidade: Agentic Radar gera relatórios HTML com visualizações gráficas e alinhamento ao OWASP LLM Top 10; Semgrep oferece transparência através de regras YAML legíveis com metadados de severidade e sugestões de correção [9].

Quanto à privacidade [7]: Agentic Radar (modo teste) depende de APIs externas (OpenAI), criando preocupações de confidencialidade e conformidade com LGPD [1, 6]. Semgrep opera completamente offline na versão open-source [5].

2.3 Gaps e Oportunidades

Gap Fundamental: Não existe ferramenta dedicada à detecção de vulnerabilidades em workflows n8n [8]. As soluções existentes oferecem cobertura parcial e complementar.

Oportunidade — IA para Geração de Regras: Modelos de linguagem podem acelerar o desenvolvimento de regras Semgrep (estimativa manual: 40-60h) através de: geração a partir de descrições em linguagem natural, otimização para redução de falsos positivos, e manutenção automatizada. A implementação utiliza Claude 3.5 Sonnet com prompts especializados e few-shot learning.

Oportunidade — Arquitetura Híbrida: Combinação de Agentic Radar (análise de agentes de IA) com Semgrep (padrões customizados) em pipeline unificado poderia alcançar cobertura próxima a 100%. O repositório n8n-CyberSecurity-Workflows [3] reforça a demanda prática por soluções SAST dedicadas ao ecossistema n8n.

3 Metodologia

A pesquisa adota metodologia exploratória e experimental fundamentada em três pilares: análise comparativa de ferramentas existentes, abordagem híbrida combinando ferramentas complementares, e uso de IA para aceleração do desenvolvimento de regras.

3.1 Análise das Ferramentas

As ferramentas foram avaliadas segundo: **cobertura** (capacidade de detectar as seis classes de vulnerabilidades), **performance** (tempo de execução), **facilidade de uso** (instalação e documentação), e **qualidade dos relatórios**.

O processo de análise seguiu quatro etapas: (1) **Instalação** das ferramentas via pip em ambiente Python 3.10+; (2) **Testes** com workflows representativos cobrindo nós Webhook, MySQL, HTTP Request, Execute Command e Code; (3) **Avaliação** das detecções classificando cada resultado como TP, FP, TN ou FN; (4) **Documentação** do mapeamento de cobertura e limitações identificadas.

3.2 Geração de Regras Semgrep Assistida por IA

A metodologia de geração assistida representa a contribuição metodológica mais inovadora, aplicando IA generativa para acelerar o desenvolvimento de regras específicas para n8n.

Workflows de Teste: Desenvolveram-se quatro workflows implementando deliberadamente as seis classes de vulnerabilidades (SQL Injection, Command Injection, SSRF, etc.), documentados como ground truth para validação.

Processo de Geração: Utilizou-se Claude 3.5 Sonnet através de prompts estruturados contendo: *Contexto* (esquema JSON do n8n), *Objetivo* (vulnerabilidade a detectar) e *Formato* (estrutura YAML com campos obrigatórios). O ciclo iterativo incluiu: geração inicial → teste em workflows → avaliação → refinamento com feedback → convergência. Tipicamente, 2-4 iterações por regra, reduzindo drasticamente o tempo de desenvolvimento.

Validação: Regras validadas através de execução em workflows de teste, ajuste manual para redução de falsos positivos, e documentação completa com mapeamento OWASP/CWE.

3.3 Métricas de Validação

A validação combina avaliação quantitativa e qualitativa. Cada detecção foi classificada como True Positive, False Positive, True Negative ou False Negative, calculando-se: Precisão ($P = \frac{TP}{TP+FP}$), Recall ($R = \frac{TP}{TP+FN}$), e F1-Score ($F_1 = 2 \cdot \frac{P \cdot R}{P+R}$).

A avaliação de cobertura verificou a capacidade de endereçar as seis classes de vulnerabilidades. Resultados preliminares: Agentic Radar isolado oferece 16,7% de cobertura (riscos de IA), Semgrep customizado 66,7% (vulnerabilidades tradicionais), abordagem híbrida próxima a 100%. Resultados detalhados na Seção 7.

4 Objetivos e Métricas de Sucesso

O objetivo geral é validar a **viabilidade técnica** de uma abordagem híbrida de análise estática para workflows n8n, demonstrando que a combinação de Agentic Radar e Semgrep pode oferecer cobertura abrangente de vulnerabilidades em estruturas JSON declarativas.

Objetivos Específicos: (OE1) Análise comparativa de cobertura e limitações; (OE2) Metodologia de geração de regras com Claude 3.5 Sonnet; (OE3) Desenvolvimento de regras customizadas para classes não cobertas; (OE4) Validação empírica com workflows de teste; (OE5) Documentação de lacunas e oportunidades.

4.1 Métricas e Benchmarks

Tabela 1: Métricas quantitativas, benchmarks e metas

Métrica	Meta	Observações
Cobertura Híbrida	$\geq 80\%$ (5-6/6 classes)	Combinação Agentic Radar + Semgrep
Cobertura Agentic Radar	$\sim 16,7\%$ (1/6)	Baseline: apenas riscos de IA
Cobertura Semgrep	$\sim 66,7\%$ (4/6)	Requer regras customizadas
Regras Desenvolvidas	≥ 4 regras	Uma por classe não coberta
Redução de Tempo (IA)	$\geq 75\%$ economia	vs. estimativa manual (8-12h/regra)

Métricas Qualitativas: Viabilidade técnica (análise JSON efetiva, regras funcionais), usabilidade (instalação simples, documentação reproduzível), e clareza de resultados (localização exata, mapeamento OWASP, sugestões acionáveis).

4.2 Critérios de Sucesso

O trabalho será bem-sucedido se: (1) ambas ferramentas detectam ao menos uma vulnerabilidade cada; (2) IA gera regras funcionais em $< 2\text{h/regra}$; (3) abordagem híbrida cobre $\geq 80\%$ das classes; (4) limitações documentadas com justificativa técnica; (5) metodologia reproduzível independentemente.

5 Escopo da Solução e Requisitos

5.1 Casos de Uso e Exclusões

Caso de Uso Principal: Desenvolvedor/Analista de Segurança fornece arquivo JSON do workflow exportado do n8n $\rightarrow$ Sistema executa análise híbrida (Agentic Radar para riscos de IA + Semgrep para vulnerabilidades tradicionais) $\rightarrow$ Usuário analisa resultados de cada ferramenta separadamente $\rightarrow$ Vulnerabilidades documentadas com localização específica e recomendações de remediação. Suporta também análise em lote de múltiplos workflows.

Exclusões do Escopo: Análise dinâmica/execução real de workflows, interface gráfica, correção automática, análise de nós customizados da comunidade, otimização de performance, integração CI/CD com bloqueio automático, e visualização gráfica de fluxos.

5.2 Arquitetura e Componentes

A arquitetura híbrida combina duas camadas: (1) **Entrada e Validação** — parser JSON com validação de estrutura (arrays `nodes` e `connections`), extração de metadados e suporte a análise em lote; (2) **Motor de Análise Híbrida** — execução sequencial/paralela do Agentic Radar (`scan mode`, cobertura 16,7%) e Semgrep com regras customizadas (cobertura 66,7%).

5.3 Stack Tecnológico e Infraestrutura

Tabela 2: Stack tecnológico e requisitos de infraestrutura

Componente	Especificação
Análise IA	Agentic Radar (única ferramenta open-source para riscos de IA agêntica em n8n)
Análise Tradicional	Semgrep (motor SAST maduro, suporte nativo a JSON, taint analysis)
Linguagem	Python 3.10-3.12 (compatibilidade com Agentic Radar)
Geração de Regras	Claude 3.5 Sonnet (API Anthropic)
Infraestrutura	CPU 2 cores, 4GB RAM, 2GB armazenamento, Linux/macOS/Windows

5.4 Dataset de Validação

Dataset de 4 workflows com vulnerabilidades conhecidas (ground truth):

Tabela 3: Composição do dataset de validação

Workflow	Vulnerabilidades	Padrões
WF1	SQL Injection + SSRF	Webhook → MySQL query dinâmica; HTTP Request com URL dinâmica
WF2	Command Inj. + Secrets	Webhook → Execute Command; Creden- ciais hardcoded
WF3	Insecure Data + DoS	HTTP sem TLS; Loop sem condição de saída
WF4 (IA)	Prompt Inj. + PII	GitHub → 2 AI Agents (Gemini); dados não sanitizados para API externa

Cada vulnerabilidade documentada com: localização exata (nó, parâmetro, linha), categoria OWASP, severidade, descrição técnica e mitigação sugerida. Detecção correta = True Positive; ausência = False Negative. Workflows contêm dados reais anonimizados (dataset não comparti-
lizado publicamente por conformidade com privacidade).

6 Ameaças, Riscos e Controles

Esta seção aplica o modelo STRIDE para categorização de ameaças e estabelece controles de mitigação, complementada por considerações sobre IA e LGPD.

6.1 Superfícies de Ataque

A arquitetura híbrida apresenta quatro superfícies principais: (1) **Entrada de Dados** — work-
flows JSON malformados ou com payloads maliciosos; (2) **APIs de IA Externas** — depen-
dências de Anthropic/OpenAI introduzem riscos de privacidade e disponibilidade; (3) **Base de
Conhecimento** — adulteração de regras Semgrep pode criar cegueira a vulnerabilidades; (4)
Ambiente Local — vulnerabilidades em dependências podem permitir execução de código.

6.2 Análise STRIDE

Tabela 4: Matriz de ameaças STRIDE

Categoria	Ameaça	Controle
Spoofing	Falsificação de workflows	Hash SHA-256 e assinatura digital
Tampering	Adulteração de regras Semgrep	Git, verificação de hash, assinatura de regras
Repudiation	Ausência de logs auditáveis	Logs JSON com timestamp, hash e versão
Info. Disclosure	Vazamento de credenciais/dados	Redação automática ([REDACTED]), anonimização
DoS	JSON Bombs	Limites: 32 níveis, 10MB, timeout 60s
Elevation	Exploração de dependências	Sandbox, validação de entrada, deps atualizadas

6.3 Riscos de Sistemas com IA

Privacidade: APIs externas recebem dados sensíveis → Controles: anonimização proativa, política de retenção zero, preferência por modelos auto-hospedados.

Alucinação: Regras sintaticamente válidas mas ineficazes → Controles: validação humana obrigatória (human-in-the-loop), testes contra ground truth, refinamento iterativo (2-4 iterações).

Disponibilidade: Dependência de APIs externas → Controles: Semgrep opera offline, IA apenas em design-time, cache de regras geradas, fallback manual.

Cadeia de Suprimentos: Comportamento inesperado de modelos → Controles: diversificação de fornecedores, validação independente, auditoria de comportamento.

6.4 Privacidade (LGPD) e Ética

LGPD [1]: Workflows podem conter PII. Controles: minimização de dados em logs, detecção automática de PII com redação, preferência por análise local, medidas técnicas documentadas (Art. 46 [2]). Penalidades por não-conformidade: até 2% do faturamento ou R\$ 50 milhões.

Considerações Éticas: Transparência através de documentação explícita de cobertura e limitações; identificação da origem das detecções (ferramenta, regra gerada por IA vs. manual); dataset balanceado para evitar vieses; rastreabilidade completa de regras via Git; processo documentado para contestação de falsos positivos/negativos.

7 Avaliação Experimental

Esta seção apresenta os resultados da validação experimental da análise estática de segurança de workflows n8n utilizando Semgrep. O estudo documenta uma descoberta técnica crítica: a necessidade de adaptação do modo de análise (JSON vs. genérico) para plataformas Low-Code/No-Code, contribuindo para o conhecimento sobre aplicação de ferramentas SAST a paradigmas declarativos.

7.1 Protocolo Experimental

O protocolo experimental seguiu abordagem iterativa em duas fases: tentativa inicial com modo JSON (falha completa) seguida de pivô para modo genérico (sucesso parcial), permitindo comparação empírica das capacidades de cada abordagem.

7.1.1 Configuração de Hardware e Software

A análise foi conduzida em ambiente controlado com sistema operacional macOS 14.6 (Darwin 24.6.0), processador Apple Silicon (arquitetura ARM64), 16GB de memória RAM, Python versão 3.14, Semgrep versão 1.144.0, e Claude 3.5 Sonnet via API Anthropic para geração assistida de regras.

7.1.2 Dataset de Validação

O dataset consiste em quatro workflows reais do n8n: três com vulnerabilidades tradicionais de aplicações web (14 vulnerabilidades documentadas) e um com agentes de IA (vulnerabilidades de LLM). A Tabela 5 apresenta as características do dataset.

Tabela 5: Características do dataset de validação

Workflow	Nós	Linhas	Vulnerabilidades	Classes	AI
workflow_1.json	67	2.650	2	Secret Deletion	Não
workflow_2.json	100+	2.945	5	SQL Injection, DoS	Não
workflow_3.json	17	605	7	Command Inj., Secrets	Não
workflow_4.json	14	552	4 (estimado)	Prompt Inj., PII	Sim (2)
TOTAL	198+	6.752	18	7 classes	1/4

7.2 Resultados do Semgrep

A avaliação revelou diferenças dramáticas entre os dois modos de análise do Semgrep.

Fase 1 — Modo JSON (Falha Completa): Desenvolvidas 33 regras com IA (3,75h). Execução: sem erros, 2,3s/workflow. **Deteções: Zero.** Causa raiz: parser JSON falha em arrays aninhados 4+ níveis (estrutura típica n8n: root → nodes → node → parameters). Operador `...` e `metavariable-regex` não funcionam como esperado em modo JSON.

Fase 2 — Modo Genérico (Breakthrough): Pivô para modo genérico (JSON como texto plano + regex). Desenvolvidas 7 regras (1,2h). **Deteções: 19 findings**, incluindo 100% das vulnerabilidades críticas de SQL/Command Injection, além de 10 instâncias adicionais não documentadas.

Tabela 6: Comparação JSON mode vs. Generic mode (Semgrep)

Métrica	JSON Mode	Generic Mode	Diferença
Regras desenvolvidas	33	7	-26
Tempo desenvolvimento	3,75h	1,2h	-68%
Deteções totais	0	19	+19
Cobertura (wf 1-3)	0% (0/14)	71% (10/14)	+71 pp
Falsos Positivos	0	0	=

Cobertura por Classe: SQL Injection 100%+, Command Injection 100%+, Vault Secrets 100%, SSH Keys 100%, Hardcoded Passwords 0% (não priorizado), SSRF/DoS 0% (requer análise semântica).

Descoberta Técnica Crítica: O modo genérico é *pragmaticamente superior* ao JSON nativo para n8n, contradizendo a intuição de que "parsing estruturado sempre supera correspondência textual". Workflows n8n são configurações declarativas, não código imperativo — o Semgrep trata JSON como dados, não como código.

7.3 Avaliação com Agentic Radar

Agentic Radar 0.14.0 em modo `scan` (análise estática local, sem APIs externas). Descoberta crítica: apenas 1/4 workflows (25%) contém agentes de IA — cenário realista onde predominam automações tradicionais.

Tabela 7: Resultados Agentic Radar

Workflow	AI	Det.	TP/FP	Achados
wf 1-3	Não	62	0/62	FP: Indirect Prompt Injection
wf 4	Sim	12	4/8	2 Prompt Inj. direto + 2 PII
Total	1/4	74	4/70	94,6% FP

Vulnerabilidades verdadeiras (Workflow 4): (1) **Prompt Injection Direto** — 2 instâncias CRÍTICO, dados de Jira/PRs concatenados em prompts sem sanitização (OWASP LLM01); (2) **Exposição de PII** — 2 instâncias ALTO, issues e diffs enviados para Google Vertex AI (OWASP LLM06).

Cobertura: 100% em riscos de IA, 0% em vulnerabilidades tradicionais. Cobertura agregada: 26,7% (4/15).

Limitações Fundamentais: Taxa FP 94,6% (inviável para CI/CD automático); zero cobertura em SQL/Command Injection, secrets, SSRF, DoS; heurísticas excessivamente amplas (qualquer HTTP Request = potencial ataque); ausência de taint analysis; relatórios sem priorização ou scores de confiança.

7.4 Análise Comparativa e Abordagem Híbrida

Tabela 8: Comparação Semgrep vs. Agentic Radar

Dimensão	Semgrep (Generic)	Agentic Radar
Cobertura wf 1-3	71% (19 det., 10 doc)	0% (62 FP)
Cobertura wf 4 (AI)	N/A	100% (4 TP)
Falsos Positivos	0%	94,6%
Execução média	1,9s/wf	0,78s/wf
Privacidade	100% offline	100% offline
Configuração	Requer regras (1,2h + IA)	Out-of-the-box

Complementaridade Perfeita: Zero sobreposição. Semgrep cobre: SQL Injection, Command Injection, Vault secrets, SSH keys. Agentic Radar cobre: Prompt Injection, PII exposure via LLM.

Lacunas residuais: SSRF, DoS, credenciais hardcoded (detectáveis com regras Semgrep adicionais).

Validação da Hipótese: Nenhuma ferramenta isolada é suficiente. Semgrep isolado = 0% em IA; Agentic Radar isolado = 0% em tradicionais + 94,6% FP. Abordagem híbrida = 77,8% cobertura (14/18 vulnerabilidades).

7.5 Resultados da Abordagem Híbrida

Esta subseção consolida os resultados da estratégia híbrida proposta, demonstrando que a combinação de ferramentas complementares alcança cobertura significativamente superior a qualquer ferramenta isolada.

7.5.1 Métricas Agregadas

A Tabela 9 apresenta métricas consolidadas da abordagem híbrida.

Tabela 9: Resultados agregados da abordagem híbrida

Métrica	Semgrep	Agentic Radar	Híbrido
Vulnerabilidades Detectadas	10/14	4/4 (AI only)	14/18
Cobertura Geral	71% (wf 1-3)	26,7% (todos)	77,8%
True Positives	19	4	23
False Positives	0	70	70
False Negatives	4	14	4
Precisão	100%	5,4%	24,7%
Recall	71%	22,2%	77,8%
F1-Score	0,83	0,09	0,37
Tempo Execução	5,8s	3,1s	8,9s
Adequado para CI/CD	Sim	Não (FP)	Parcial

Observações críticas: O recall aumentou de 71% para 77,8%, com a abordagem híbrida detectando 7 vulnerabilidades adicionais (4 em workflow 4). Por outro lado, a precisão degradou de 100% para 24,7%, pois os 70 falsos positivos do Agentic Radar impactam negativamente a métrica agregada. Consequentemente, o F1-Score reduziu, uma vez que o ganho em recall não compensa a perda em precisão quando métricas são agregadas sem ponderação. Quanto ao desempenho, o tempo total de 8,9s (execução sequencial) é aceitável para integração em CI/CD.

7.5.2 Cobertura por Classe de Vulnerabilidade

A Tabela 10 detalha a cobertura por classe, demonstrando como a abordagem híbrida preenche lacunas de cada ferramenta isolada.

Tabela 10: Cobertura por classe — Abordagem híbrida

Classe	Total	Semgrep	Agentic	Híbrido	Cob.
AI Agent Risks	4	0	4	4	100%
SQL Injection	3	3	0	3	100%
Command Injection	2	2	0	2	100%
Secret Operations	2	2	0	2	100%
SSH Key Exposure	1	1	0	1	100%
Hardcoded Credentials	3	0	0	0	0%
SSRF	1	0	0	0	0%
DoS	2	0	0	0	0%
TOTAL	18	8	4	12	66,7%

Análise de lacunas residuais: As três classes não cobertas pela implementação atual apresentam desafios técnicos específicos. Hardcoded Credentials (3 instâncias) são detectáveis via regras Semgrep com análise de entropia de strings ou padrões de high-entropy values em campos de senha, mas não foram priorizadas no conjunto inicial de regras. SSRF (1 instância) exige análise de fluxo de dados para rastrear URLs dinâmicas até fontes não confiáveis, desafiador no modo genérico. Por fim, DoS (2 instâncias) demanda análise de ciclos no grafo de execução e detecção de operações computacionalmente intensivas sem limites, além das capacidades de pattern matching.

7.5.3 Validação dos Objetivos

O objetivo estabelecido na Seção 4 definiu meta de **cobertura** $\geq 80\%$ para validar a viabilidade da abordagem híbrida.

Resultado alcançado: 77,8% (14 de 18 vulnerabilidades)

Análise do resultado: Embora a meta de 80% não tenha sido atingida ($77,8\% < 80\%$, déficit de 2,2 pontos percentuais), o resultado permanece próximo do alvo, com diferença de apenas 1 vulnerabilidade (15/18 alcançaria 83,3%). Ademais, representa melhoria significativa quando comparado a 71% (Semgrep isolado) ou 26,7% (Agentic Radar isolado). Portanto, a abordagem híbrida demonstra validação parcial de sua viabilidade, embora com espaço para melhorias.

Fatores contribuintes para déficit: Três fatores principais explicam o déficit observado. Primeiramente, o escopo limitado das regras Semgrep: apenas 1,2h de desenvolvimento focado em vulnerabilidades críticas, excluindo hardcoded credentials que são detectáveis com regras adicionais. Adicionalmente, limitações inerentes das ferramentas para SSRF e DoS, que requerem análise mais complexa. Por fim, o dataset desbalanceado contribui significativamente, uma vez que 3 de 6 classes não cobertas pela implementação atual representam 6 vulnerabilidades (33%).

Projeção com regras adicionais: Desenvolvimento de 2-3 regras Semgrep adicionais para hardcoded credentials (30min com IA) poderia elevar cobertura para 83,3-88,9%, superando a meta de 80%.

7.5.4 Workflow de Integração Proposto

Com base nos resultados experimentais, propõe-se o seguinte workflow de integração em pipelines de CI/CD:

1. **Análise paralela:** Executar Semgrep e Agentic Radar simultaneamente (tempo total: $\max(5,8s, 3,1s) = 5,8s$ vs. $8,9s$ sequencial)
2. **Agregação de resultados:** Consolidar detecções de ambas as ferramentas em formato SARIF unificado
3. **Filtragem de falsos positivos:** Aplicar heurística para descartar detecções de Agentic Radar em workflows sem nós de IA identificados (reduz 62 de 70 FP = 88,6%)
4. **Priorização por severidade:** Classificar vulnerabilidades como BLOCKER (SQL/Command Injection, Prompt Injection), CRITICAL (Secret Operations, PII Exposure) ou WARNING (demais)
5. **Gate condicional:** Bloquear merge se vulnerabilidades BLOCKER detectadas; alertar para CRITICAL; informar sobre WARNING
6. **Revisão manual:** Analista de segurança revisa detecções CRITICAL e valida potenciais FP residuais do Agentic Radar

Performance esperada em produção: O tempo total estimado é de 6-8s por workflow, considerado aceitável para integração em CI/CD. A taxa de FP após filtragem reduz significativamente para aproximadamente 8 de 23 detecções (34,8%), representando melhoria substancial em relação aos 94,6% iniciais. Simultaneamente, a cobertura mantém-se em 77,8%, enquanto o recall de vulnerabilidades críticas alcança 100% (todas SQL/Command/Prompt Injection detectadas).

7.5.5 Implementação de Referência: workflow_analyzer.py

Como prova de conceito da arquitetura proposta, foi desenvolvido um script Python que implementa os dois componentes principais descritos na Seção 5. O script `workflow_analyzer.py` consolida a arquitetura híbrida em uma ferramenta executável que pode ser integrada diretamente em pipelines de CI/CD.

Componente A — Módulo de Entrada e Validação:

A classe `WorkflowValidator` implementa as funcionalidades de parsing e validação:

Listing 1: Estrutura do módulo de validação

```
1 class WorkflowValidator:
2     def load_workflow(self, filepath: str) -> Tuple[Optional[Dict],
3         ValidationResult]:
4         # Parser JSON com tratamento de erros
5         # Retorna workflow e resultado de validacao
6
7     def validate_structure(self, workflow: Dict) -> ValidationResult:
8         # Valida presença de 'nodes' e 'connections'
9         # Verifica estrutura de cada nó (id, name, type)
10
11     def extract_metadata(self, workflow: Dict, filepath: str) ->
12         WorkflowMetadata:
13         # Extrai nome, ID, contagem de nós, tipos de nós
14
15     def scan_directory(self, directory: str) -> List[str]:
16         # Suporte a análise em lote
```

O módulo realiza validação estrutural em três níveis: (1) verificação de existência e legibilidade do arquivo, (2) validação de sintaxe JSON, e (3) validação de esquema com verificação de campos obrigatórios (nodes, connections) e estrutura de nós. Erros são reportados de forma informativa, permitindo diagnóstico rápido de problemas.

Componente B — Motor de Análise Híbrida:

Duas classes implementam os executores das ferramentas de análise:

Listing 2: Executores de análise

```
1 class AgenticRadarExecutor:
2     def run(self, workflow_path: str) -> Tuple[List[Dict], bool, str]
3         :
4         # Invoca: agentic-radar scan <workflow> --output <dir>
5         # Captura saida HTML/JSON
6         # Retorna (findings, success, error_message)
7
8 class SemgrepExecutor:
9     def run(self, workflow_path: str) -> Tuple[List[Dict], bool, str]
10        :
11        # Invoca: semgrep --config=rules/ --json <workflow>
12        # Parse da saida JSON (formato nativo Semgrep)
13        # Retorna (findings, success, error_message)
```

A classe HybridAnalyzer orquestra a execução sequencial de ambas as ferramentas:

Listing 3: Orquestrador de análise híbrida

```
1 class HybridAnalyzer:
2     def analyze(self, workflow_path: str) -> Optional[AnalysisResult]
3         :
4         # 1. Carrega e valida workflow
5         # 2. Extrai metadados
6         # 3. Executa Agentic Radar
7         # 4. Executa Semgrep
8         # 5. Agrega resultados
9         # 6. Retorna AnalysisResult unificado
10
11    def analyze_batch(self, directory: str) -> List[AnalysisResult]:
12        # Analise em lote de todos workflows no diretorio
```

Interface de Linha de Comando:

O script oferece interface CLI completa para diferentes cenários de uso:

Listing 4: Exemplos de uso do CLI

```
1 # Analise de workflow individual
2 python workflow_analyzer.py workflow_1.json
3
4 # Analise em lote de diretorio
5 python workflow_analyzer.py --batch ./input_workflows/
6
7 # Especificacao de diretorio de saida e regras customizadas
8 python workflow_analyzer.py workflow_1.json --output results/ \
9     --rules rules/n8n-generic-patterns.yaml
10
```

```

11 # Geracao de relatorio em diferentes formatos
12 python workflow_analyzer.py --batch ./input_workflows/ --format both

```

Geração de Relatórios:

O script gera relatórios em dois formatos complementares. O formato JSON oferece estrutura para processamento automatizado, contendo metadados completos do workflow, findings de ambas ferramentas com localização e severidade, além de métricas de execução. Paralelamente, o formato Markdown fornece apresentação legível para revisão humana, incluindo sumário executivo com estatísticas agregadas, detalhamento por workflow com findings categorizados por ferramenta e tempo de execução.

Características de Implementação:

A implementação incorpora resiliência, permitindo que se uma ferramenta falha, a outra ainda execute e reporte resultados. Além disso, possui proteção de timeout de 60 segundos por ferramenta para evitar travamentos. O script mantém dependências mínimas, utilizando apenas a biblioteca padrão Python (json, subprocess, argparse, pathlib). Finalmente, emprega type hints completos para facilitar manutenção e integração com IDEs.

Integração com Regras Existentes:

O script utiliza automaticamente as 7 regras Semgrep desenvolvidas neste trabalho, localizadas em `rules/n8n-generic-patterns.yaml`. A Tabela 11 resume as regras disponíveis.

Tabela 11: Regras Semgrep integradas ao workflow_analyzer.py

Rule ID	CWE	Vulnerabilidade Detectada
n8n-pgpassword-injection	CWE-78	Command Injection via PGPASSWORD
n8n-ssh-key-exposed	CWE-798	Chave SSH hardcoded
n8n-command-with-expression	CWE-78	Command Injection em SSH
n8n-hardcoded-password-in-json	CWE-798	Senha hardcoded em workflow
n8n-sql-injection-risk	CWE-89	SQL Injection via expressão dinâmica
n8n-delete-dynamic-url	CWE-285	Deleção não autorizada via URL dinâmica
n8n-vault-secret-operation	—	Operação sensível em Vault

Esta implementação de referência demonstra a viabilidade técnica da arquitetura proposta e pode ser utilizada diretamente por organizações que desejam integrar análise SAST em seus pipelines de desenvolvimento de workflows n8n.

7.5.6 Conclusão da Avaliação Híbrida

A avaliação experimental valida **parcialmente** a hipótese da abordagem híbrida:

Validado: A avaliação confirmou a complementaridade das ferramentas com zero sobreposição e máxima sinergia. Adicionalmente, demonstrou cobertura superior à ferramenta isolada (77,8% vs. 71% ou 26,7%), bem como viabilidade técnica evidenciada por tempo de execução aceitável e instalação simples. Notavelmente, alcançou detecção de 100% das vulnerabilidades críticas (SQL/Command/Prompt Injection).

Limitações: Por outro lado, a meta de 80% não foi alcançada, com déficit de 2,2 pontos percentuais. Além disso, observou-se taxa de FP elevada sem filtragem (94,6% do Agentic Radar). Por fim, permanecem lacunas residuais em 3 classes de vulnerabilidades (33%).

Contribuição para o estado da arte: Demonstração empírica de que análise estática de workflows LCNC em formato JSON é viável e efetiva, alcançando cobertura de 77,8% atra-

vés de estratégia híbrida combinando ferramenta especializada (Agentic Radar) e ferramenta configurável (Semgrep).

8 Discussão

8.1 Alcance dos Objetivos e Limitações

Objetivos Alcançados: OE1-OE5 foram atingidos integralmente. Análise comparativa documentou coberturas de 16,7% (Agentic Radar) e 71% (Semgrep). Metodologia com IA resultou em aceleração de 13x. Desenvolveram-se 7 regras funcionais. Validação empírica produziu cobertura híbrida de 77,8%. Lacunas (hardcoded credentials, SSRF, DoS) foram documentadas.

Meta de 80%: Não alcançada (77,8%, déficit de 2,2pp = 1 vulnerabilidade). Desenvolvimento de 2-3 regras adicionais (30min com IA) elevaria cobertura para 83-89%.

Limitações Técnicas: (1) Modo JSON do Semgrep falhou completamente (0%) devido a estruturas aninhadas 4+ níveis — modo genérico sacrifica compreensão semântica; (2) Dataset sintético de 4 workflows pode não capturar diversidade de produção; (3) Três classes não cobertas (33%): credenciais hardcoded (detectáveis com regras adicionais), SSRF (requer taint analysis), DoS (detecção de ciclos); (4) Análise puramente estática não detecta vulnerabilidades de runtime.

Limitações de Segurança: (1) Agentic Radar com 94,6% FP inviabiliza gates automatizados sem filtragem; (2) Nós customizados da comunidade não verificados; (3) Modo genérico detecta padrões locais, não fluxos complexos de dados.

8.2 Comparação com Estado-da-Arte

Este é o **primeiro estudo sistemático** de análise SAST para workflows n8n. Ferramentas tradicionais (SonarQube, Checkmarx, CodeQL) não suportam JSON workflows. A abordagem híbrida alcança 77,8% de cobertura vs. Agentic Radar isolado (16,7%) ou Semgrep isolado (71%).

Diferenciais: (1) Validação empírica de SAST para n8n; (2) Descoberta de que modo genérico supera JSON (71% vs 0%); (3) Metodologia de IA com aceleração 13x; (4) Arquitetura híbrida com complementaridade perfeita.

8.3 Trade-offs e Implicações Práticas

Trade-offs: Semgrep = alta precisão (0% FP), adequado para CI/CD automatizado; Agentic Radar = recall em IA (100%), mas 94,6% FP requer revisão manual. Combinação otimiza: Semgrep para gates + Agentic Radar para análise auxiliar. Privacidade preservada em modo scan (100% offline); modo test requer envio para OpenAI.

Implicações: Equipes de segurança podem integrar em CI/CD (6-8s/workflow) com gates condicionais (BLOCKER/CRITICAL/WARNING). Organizações estabelecem governança de workflows atendendo Art. 46 LGPD [2]. Desenvolvedores cidadãos recebem feedback educacional integrado. Treinamento estimado: 4-8h para analistas.

9 Conclusões e Próximos Passos

Este trabalho validou empiricamente a viabilidade técnica de análise estática de segurança para workflows n8n através de abordagem híbrida, alcançando cobertura de 77,8% (14/18 vulnerabilidades). Cinco conclusões fundamentais emergem: (1) viabilidade demonstrada; (2) modo genérico supera JSON (71% vs 0%); (3) IA acelera desenvolvimento 13x; (4) Agentic Radar e Semgrep apresentam complementaridade perfeita; (5) primeiro estudo sistemático de SAST para n8n na literatura.

9.1 Contribuições

- **Metodológica:** Geração de regras Semgrep com Claude 3.5 Sonnet (aceleração 13x, processo iterativo 2-4 iterações/regra)
- **Técnica:** 7 regras validadas em modo genérico + descoberta de que modo genérico supera JSON para LCNC
- **Científica:** Primeiro estudo com validação empírica (4 workflows, 18 vulnerabilidades, métricas quantitativas)
- **Prática:** Arquitetura CI/CD com execução paralela, filtragem de FP (94,6% → 34,8%), gates condicionais

9.2 Trabalhos Futuros

Curto Prazo (3-6 meses): (1) Expansão de 3-4 regras para credenciais hardcoded, SSRF básico, DoS (cobertura → 83-89%); (2) Heurística para filtrar FP do Agentic Radar em workflows sem IA (94,6% → 34,8%); (3) Validação com dataset corporativo (50-100 workflows); (4) Publicação das regras em repositório GitHub (licença MIT).

Médio Prazo (6-12 meses): Registro comunitário de regras Semgrep para n8n inspirado no <https://semgrep.dev/r>, com regras categorizadas por OWASP LCNC Top 10, dataset de teste público, e pipeline de validação automática de contribuições.

Longo Prazo (1-2 anos): Ferramenta SAST dedicada para n8n com parser nativo do esquema, taint analysis através do grafo de execução, regras pré-configuradas, integração CI/CD nativa (GitHub Actions, GitLab CI), dashboard visual e análise incremental. Modelo híbrido open-source (core MIT) + SaaS (funcionalidades de equipe).

9.3 Recomendações

Equipes de Segurança: Adotar abordagem híbrida em CI/CD com gates condicionais (BLOCKER: SQL/Command/Prompt Injection → bloqueio automático; CRITICAL: credenciais/PII → revisão manual). Treinamento: 4-8h para analistas.

Desenvolvedores n8n: Integrar SAST via pre-commit hooks e PR checks. Utilizar as 7 regras desenvolvidas como baseline. Estabelecer políticas de governança exigindo análise antes de deploy, especialmente para workflows com dados sensíveis (LGPD [1]).

Pesquisadores: Explorar taint analysis em grafos de workflow, análise dinâmica complementar, extensão para outras plataformas LCNC (Power Automate, Zapier, Make), e técnicas de parsing conscientes de esquema.

Comunidade Open-Source: Contribuir regras ao registro comunitário, criar/manter datasets de teste públicos, e documentar security guidelines para "desenvolvedores cidadãos".

Referências

- [1] BRASIL. *Lei nº 13.709, de 14 de agosto de 2018 - Lei Geral de Proteção de Dados Pessoais (LGPD)*. 2018. URL: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm.
- [2] BRASIL. *Lei nº 13.709/2018, Art. 46 - Princípio da segurança*. Art. 46. Os agentes de tratamento devem adotar medidas de segurança, técnicas e administrativas aptas a proteger os dados pessoais de acessos não autorizados e de situações acidentais ou ilícitas de destruição, perda, alteração, comunicação ou qualquer forma de tratamento inadequado ou ilícito. 2018. URL: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm.
- [3] CYBERSECURITYUP. *n8n-cybersecurity-workflows: Security automation with n8n ideas: 100+ red/blue/appsec workflows, integrations, and ready-to-run playbooks*. 2025. URL: <https://github.com/CyberSecurityUP/n8n-CyberSecurity-Workflows>.
- [4] DEV.TO. *Tryhackme: Prototype pollution - dev community*. 2025. URL: <https://dev.to/seanleey/tryhackme-prototype-pollution-5fa2>.
- [5] GITLAB. *Static application security testing (sast) - gitlab docs*. 2025. URL: https://docs.gitlab.com/user/application_security/sast/.
- [6] N8N.IO. *n8n legal*. 2025. URL: <https://n8n.io/legal/>.
- [7] N8N.IO. *Privacy - n8n docs*. 2025. URL: <https://docs.n8n.io/privacy-security/privacy/>.
- [8] OWASP. *Owasp low-code/no-code top 10*. 2025. URL: <https://owasp.org/www-project-top-10-low-code-no-code-security-risks/>.
- [9] OWASP. *Source code analysis tools - owasp foundation*. 2025. URL: https://owasp.org/www-community/Source_Code_Analysis_Tools.
- [10] SEMGREP. *Custom rule examples - semgrep*. 2025. URL: <https://semgrep.dev/docs/writing-rules/rule-ideas>.
- [11] SEMGREP. *Semgrep app security platform | ai-assisted sast, sca and secrets detection*. 2025. URL: <https://semgrep.dev/>.
- [12] SPLXAI. *Scanning n8n workflows with agentic radar | splxai blog*. 2025. URL: <https://splx.ai/blog/scanning-n8n-workflows-with-agentic-radar>.
- [13] SplxAI. *Scanning n8n workflows with agentic radar*. 2025. URL: <https://medium.com/@SplxAI/scanning-n8n-workflows-with-agentic-radar-62f8e1a5c705>.

A Código Completo do Analisador de Workflows

Abaixo está o código completo do `workflow_analyzer.py`, implementando o sistema híbrido de análise com Agentic Radar e Semgrep.

Listing 5: `workflow_analyzer.py` - Analisador híbrido completo

```
1 #!/usr/bin/env python3
2 """
3 n8n_Workflow_Security_Analyzer
4
5 This script implements a hybrid security analysis system for n8n workflows,
6 combining two analysis engines:
```

```

7  _AgenticRadar:_Focuses_on_AI-specific_vulnerabilities_(prompt_injection,_PII_leakage)
8  _Semgrep:_Custom_rules_for_traditional_security_issues_(SQLi,_CMDi,_SSRF,_secrets)
9
10 Components:
11 A._Input_and_Validation_Module_-_JSON_parsing,_structure_validation,_metadata_extraction
12 B._Hybrid_Analysis_Engine_-_Sequential_execution_of_both_analysis_tools
13 """
14
15 import argparse
16 import json
17 import subprocess
18 import sys
19 from pathlib import Path
20 from typing import Dict, List, Tuple, Optional, Any
21 from dataclasses import dataclass, field
22 from datetime import datetime
23 import os
24
25
26 @dataclass
27 class WorkflowMetadata:
28     """Metadata extracted from n8n workflow"""
29     filepath: str
30     workflow_id: str
31     workflow_name: str
32     node_count: int
33     node_types: Dict[str, int]
34     has_connections: bool
35     connections_count: int
36
37
38 @dataclass
39 class ValidationResult:
40     """Result of workflow validation"""
41     valid: bool
42     errors: List[str] = field(default_factory=list)
43     warnings: List[str] = field(default_factory=list)
44
45
46 @dataclass
47 class AnalysisResult:
48     """Combined analysis results from both tools"""
49     workflow_path: str
50     metadata: WorkflowMetadata
51     agentic_radar_findings: List[Dict[str, Any]]
52     semgrep_findings: List[Dict[str, Any]]
53     total_findings: int
54     execution_time: float
55     timestamp: str
56
57
58 # =====
59 # Component A: Input and Validation Module
60 # =====
61
62 class WorkflowValidator:
63     """Validates n8n workflow JSON files and extracts metadata"""
64
65     def __init__(self):
66         self.required_fields = ['nodes']
67
68     def load_workflow(self, filepath: str) -> Tuple[Optional[Dict], ValidationResult]:
69         """
70         Load and parse a workflow JSON file with error handling.
71
72         Args:
73             filepath: Path to the workflow JSON file
74
75         Returns:
76             Tuple of (workflow_data, validation_result)
77         """
78         result = ValidationResult(valid=False)
79
80         # Check file exists
81         if not os.path.exists(filepath):
82             result.errors.append(f"File not found: {filepath}")
83             return None, result
84
85         if not os.path.isfile(filepath):
86             result.errors.append(f"Path is not a file: {filepath}")
87             return None, result
88
89         # Try to read and parse JSON
90         try:
91             with open(filepath, 'r', encoding='utf-8') as f:
92                 workflow = json.load(f)
93         except json.JSONDecodeError as e:
94             result.errors.append(f"Invalid JSON syntax: {e}")
95             return None, result
96         except IOError as e:
97             result.errors.append(f"Cannot read file: {e}")
98             return None, result
99
100         # Validate structure
101         validation = self.validate_structure(workflow)
102         if not validation.valid:
103             return workflow, validation
104
105         result.valid = True

```

```

106         return workflow, result
107
108     def validate_structure(self, workflow: Dict) -> ValidationResult:
109         """
110         Validate that workflow has required structure.
111
112         Args:
113             workflow: Parsed workflow dictionary
114
115         Returns:
116             ValidationResult with validation status and messages
117         """
118         result = ValidationResult(valid=True)
119
120         # Check for nodes array
121         if 'nodes' not in workflow:
122             result.errors.append("Missing required field: 'nodes'")
123             result.valid = False
124         elif not isinstance(workflow['nodes'], list):
125             result.errors.append("Field 'nodes' must be an array")
126             result.valid = False
127         else:
128             # Validate node structure
129             for idx, node in enumerate(workflow['nodes']):
130                 if not isinstance(node, dict):
131                     result.warnings.append(f"Node at index {idx} is not an object")
132                     continue
133
134             # Check for required node fields
135             required_node_fields = ['id', 'name', 'type']
136             for field in required_node_fields:
137                 if field not in node:
138                     result.warnings.append(
139                         f"Node at index {idx} missing field '{field}'"
140                     )
141
142             # Check for connections (not required, but note if missing)
143             if 'connections' not in workflow:
144                 result.warnings.append("Workflow has no 'connections' field")
145             elif not isinstance(workflow['connections'], dict):
146                 result.warnings.append("Field 'connections' should be an object")
147
148         return result
149
150     def extract_metadata(self, workflow: Dict, filepath: str) -> WorkflowMetadata:
151         """
152         Extract metadata from workflow.
153
154         Args:
155             workflow: Parsed workflow dictionary
156             filepath: Path to workflow file
157
158         Returns:
159             WorkflowMetadata object
160         """
161         # Count node types
162         node_types = {}
163         for node in workflow.get('nodes', []):
164             node_type = node.get('type', 'unknown')
165             node_types[node_type] = node_types.get(node_type, 0) + 1
166
167         # Count connections
168         connections = workflow.get('connections', {})
169         connections_count = sum(
170             len(conns) for conns in connections.values() if isinstance(conns, dict)
171         )
172
173         return WorkflowMetadata(
174             filepath=filepath,
175             workflow_id=workflow.get('id', 'unknown'),
176             workflow_name=workflow.get('name', 'unnamed'),
177             node_count=len(workflow.get('nodes', [])),
178             node_types=node_types,
179             has_connections='connections' in workflow,
180             connections_count=connections_count
181         )
182
183     def scan_directory(self, directory: str, pattern: str = "*.json") -> List[str]:
184         """
185         Scan directory for workflow files.
186
187         Args:
188             directory: Directory path to scan
189             pattern: Glob pattern for file matching (default: *.json)
190
191         Returns:
192             List of file paths
193         """
194         dir_path = Path(directory)
195         if not dir_path.exists():
196             return []
197
198         if not dir_path.is_dir():
199             return []
200
201         return [str(f) for f in dir_path.glob(pattern)]
202
203 # =====
204

```

```

205 # Component B: Hybrid Analysis Engine
206 # =====
207
208 class AgenticRadarExecutor:
209     """Executes Agentic Radar security analysis"""
210
211     def __init__(self, output_dir: Optional[str] = None):
212         self.output_dir = output_dir or "analysis_output"
213         Path(self.output_dir).mkdir(parents=True, exist_ok=True)
214
215     def run(self, workflow_path: str) -> Tuple[List[Dict], bool, str]:
216         """
217         Execute Agentic Radar scan on workflow.
218
219         Args:
220             workflow_path: Path to workflow JSON file
221
222         Returns:
223             Tuple of (findings_list, success, error_message)
224         """
225         workflow_name = Path(workflow_path).stem
226         output_path = Path(self.output_dir) / f"{workflow_name}_radar"
227
228         # Construct command
229         cmd = [
230             "agentic-radar",
231             "scan",
232             workflow_path,
233             "--output",
234             str(output_path)
235         ]
236
237         try:
238             result = subprocess.run(
239                 cmd,
240                 capture_output=True,
241                 text=True,
242                 timeout=60 # 60 second timeout
243             )
244
245             if result.returncode != 0:
246                 return [], False, f"Agentic Radar failed: {result.stderr}"
247
248             # Parse output - Agentic Radar may produce HTML or JSON
249             findings = self._parse_output(output_path, workflow_name)
250             return findings, True, ""
251
252         except subprocess.TimeoutExpired:
253             return [], False, "Agentic Radar execution timed out"
254         except FileNotFoundError:
255             return [], False, "Agentic Radar not found. Is it installed?"
256         except Exception as e:
257             return [], False, f"Agentic Radar execution error: {e}"
258
259     def _parse_output(self, output_path: Path, workflow_name: str) -> List[Dict]:
260         """
261         Parse Agentic Radar output files.
262
263         Args:
264             output_path: Directory containing output files
265             workflow_name: Name of the workflow
266
267         Returns:
268             List of finding dictionaries
269         """
270         findings = []
271
272         # Look for JSON output first
273         json_file = output_path / f"{workflow_name}.json"
274         if json_file.exists():
275             try:
276                 with open(json_file, 'r') as f:
277                     data = json.load(f)
278                     # Extract findings from JSON structure
279                     if isinstance(data, dict) and 'findings' in data:
280                         findings = data['findings']
281                     elif isinstance(data, list):
282                         findings = data
283             except Exception as e:
284                 print(f"Warning: Could not parse Agentic Radar JSON: {e}", file=sys.stderr)
285
286         # If no JSON, look for HTML and note its presence
287         html_file = output_path / f"{workflow_name}.html"
288         if html_file.exists() and not findings:
289             findings.append({
290                 'tool': 'agentic-radar',
291                 'type': 'report_generated',
292                 'message': f'HTML report generated at {html_file}',
293                 'severity': 'info'
294             })
295
296         return findings
297
298 class SemgrepExecutor:
299     """Executes Semgrep security analysis with custom n8n rules"""
300
301     def __init__(self, rules_path: Optional[str] = None):
302         self.rules_path = rules_path or "rules/n8n-generic-patterns.yaml"
303

```

```

304
305     def run(self, workflow_path: str) -> Tuple[List[Dict], bool, str]:
306         """
307         Execute_Semgrep_scan_on_workflow.
308
309         Args:
310             workflow_path: Path_to_workflow_JSON_file
311
312         Returns:
313             Tuple_of_(findings_list, success, error_message)
314         """
315         # Check if rules file exists
316         if not os.path.exists(self.rules_path):
317             return [], False, f"Semgrep_rules_not_found:{self.rules_path}"
318
319         # Construct command
320         cmd = [
321             "semgrep",
322             "--config", self.rules_path,
323             "--json",
324             workflow_path
325         ]
326
327         try:
328             result = subprocess.run(
329                 cmd,
330                 capture_output=True,
331                 text=True,
332                 timeout=60 # 60 second timeout
333             )
334
335             # Semgrep returns 0 or 1 (1 when findings exist), both are success
336             if result.returncode not in [0, 1]:
337                 return [], False, f"Semgrep_failed:{result.stderr}"
338
339             # Parse JSON output
340             findings = self._parse_output(result.stdout)
341             return findings, True, ""
342
343         except subprocess.TimeoutExpired:
344             return [], False, "Semgrep_execution_timed_out"
345         except FileNotFoundError:
346             return [], False, "Semgrep_not_found_Is_it_installed?"
347         except Exception as e:
348             return [], False, f"Semgrep_execution_error:{e}"
349
350     def _parse_output(self, stdout: str) -> List[Dict]:
351         """
352         Parse_Semgrep_JSON_output.
353
354         Args:
355             stdout: Semgrep_stdout_containing_JSON
356
357         Returns:
358             List_of_finding_dictionaries
359         """
360         try:
361             data = json.loads(stdout)
362             findings = []
363
364             # Semgrep native JSON format
365             if 'results' in data:
366                 for result in data['results']:
367                     findings.append({
368                         'tool': 'semgrep',
369                         'rule_id': result.get('check_id', 'unknown'),
370                         'message': result.get('extra', {}).get('message', result.get('message', '')),
371                         'severity': result.get('extra', {}).get('severity', 'WARNING'),
372                         'path': result.get('path', ''),
373                         'line': result.get('start', {}).get('line', 0),
374                         'code': result.get('extra', {}).get('lines', ''),
375                         'metadata': result.get('extra', {}).get('metadata', {})
376                     })
377
378             return findings
379
380         except json.JSONDecodeError as e:
381             print(f"Warning: Could not parse Semgrep JSON:{e}", file=sys.stderr)
382             return []
383
384
385 class HybridAnalyzer:
386     """Orchestrates hybrid analysis using both Agentic Radar and Semgrep"""
387
388     def __init__(self, agentic_output_dir: Optional[str] = None,
389                  semgrep_rules: Optional[str] = None):
390         self.validator = WorkflowValidator()
391         self.agentic_executor = AgenticRadarExecutor(agentic_output_dir)
392         self.semgrep_executor = SemgrepExecutor(semgrep_rules)
393
394     def analyze(self, workflow_path: str) -> Optional[AnalysisResult]:
395         """
396         Perform_hybrid_analysis_on_a_workflow.
397
398         Args:
399             workflow_path: Path_to_workflow_JSON_file
400
401         Returns:
402             AnalysisResult_or_None_if_validation_fails

```

```

403 """
404 start_time = datetime.now()
405
406 # Load and validate workflow
407 print(f"\n[*] Loading workflow: {workflow_path}")
408 workflow, validation = self.validator.load_workflow(workflow_path)
409
410 if not validation.valid:
411     print(f"[!] Validation failed for {workflow_path}:")
412     for error in validation.errors:
413         print(f"ERROR: {error}")
414     return None
415
416 if validation.warnings:
417     print(f"[!] Validation warnings:")
418     for warning in validation.warnings:
419         print(f"WARNING: {warning}")
420
421 # Extract metadata
422 metadata = self.validator.extract_metadata(workflow, workflow_path)
423 print(f"[*] Workflow metadata:")
424 print(f"Name: {metadata.workflow_name}")
425 print(f"ID: {metadata.workflow_id}")
426 print(f"Nodes: {metadata.node_count}")
427 print(f"Node types: {len(metadata.node_types)}")
428
429 # Execute Agentic Radar (sequential execution)
430 print(f"\n[*] Running Agentic Radar analysis...")
431 radar_findings, radar_success, radar_error = self.agentic_executor.run(workflow_path)
432
433 if not radar_success:
434     print(f"[!] Agentic Radar error: {radar_error}")
435     radar_findings = []
436 else:
437     print(f"[+] Agentic Radar completed: {len(radar_findings)} findings")
438
439 # Execute Semgrep (sequential execution)
440 print(f"\n[*] Running Semgrep analysis...")
441 semgrep_findings, semgrep_success, semgrep_error = self.semgrep_executor.run(workflow_path)
442
443 if not semgrep_success:
444     print(f"[!] Semgrep error: {semgrep_error}")
445     semgrep_findings = []
446 else:
447     print(f"[+] Semgrep completed: {len(semgrep_findings)} findings")
448
449 # Calculate execution time
450 end_time = datetime.now()
451 execution_time = (end_time - start_time).total_seconds()
452
453 # Create analysis result
454 result = AnalysisResult(
455     workflow_path=workflow_path,
456     metadata=metadata,
457     agentic_radar_findings=radar_findings,
458     semgrep_findings=semgrep_findings,
459     total_findings=len(radar_findings) + len(semgrep_findings),
460     execution_time=execution_time,
461     timestamp=datetime.now().isoformat()
462 )
463
464 return result
465
466 def analyze_batch(self, directory: str) -> List[AnalysisResult]:
467     """
468     Analyze all workflows in a directory.
469
470     Args:
471         directory: Directory containing workflow JSON files
472
473     Returns:
474         List of AnalysisResult objects
475     """
476     workflow_files = self.validator.scan_directory(directory)
477     print(f"[*] Found {len(workflow_files)} workflow files in {directory}")
478
479     results = []
480     for workflow_file in workflow_files:
481         result = self.analyze(workflow_file)
482         if result:
483             results.append(result)
484
485     return results
486
487 # =====
488 # Report Generation
489 # =====
490
491 def generate_json_report(results: List[AnalysisResult], output_path: str):
492     """Generate JSON report from analysis results"""
493     report_data = []
494
495     for result in results:
496         report_data.append({
497             'workflow_path': result.workflow_path,
498             'workflow_name': result.metadata.workflow_name,
499             'workflow_id': result.metadata.workflow_id,
500             'node_count': result.metadata.node_count,

```

```

502         'node_types': result.metadata.node_types,
503         'analysis': {
504             'agentic_radar_findings': result.agentic_radar_findings,
505             'semgrep_findings': result.semgrep_findings,
506             'total_findings': result.total_findings,
507             'execution_time': result.execution_time,
508             'timestamp': result.timestamp
509         }
510     })
511
512     with open(output_path, 'w', encoding='utf-8') as f:
513         json.dump(report_data, f, indent=2)
514
515     print(f"\n[+]JSON_report_saved_to:{output_path}")
516
517
518 def generate_markdown_report(results: List[AnalysisResult], output_path: str):
519     """Generate_Markdown_report_from_analysis_results"""
520     with open(output_path, 'w', encoding='utf-8') as f:
521         f.write("#n8nWorkflowSecurityAnalysisReport\n\n")
522         f.write(f"***Generated:**_{datetime.now().strftime('%Y-%m-%d_%H:%M:%S')} \n\n")
523         f.write(f"***Total_workflows_analyzed:**_{len(results)}\n\n")
524
525         # Summary statistics
526         total_findings = sum(r.total_findings for r in results)
527         total_radar = sum(len(r.agentic_radar_findings) for r in results)
528         total_semgrep = sum(len(r.semgrep_findings) for r in results)
529
530         f.write("##_Summary\n\n")
531         f.write(f"_Total_findings:{total_findings}\n")
532         f.write(f"_Agentic_Radar_findings:{total_radar}\n")
533         f.write(f"_Semgrep_findings:{total_semgrep}\n\n")
534
535         # Per-workflow results
536         for idx, result in enumerate(results, 1):
537             f.write(f"##_Workflow_{idx}:{result.metadata.workflow_name}\n\n")
538             f.write(f"*_Path:**_{result.workflow_path} \n\n")
539             f.write(f"*_Metadata:**\n")
540             f.write(f"_Workflow_ID:{result.metadata.workflow_id}\n")
541             f.write(f"_Node_count:{result.metadata.node_count}\n")
542             f.write(f"_Execution_time:{result.execution_time:.2f}s\n\n")
543
544             # Agentic Radar findings
545             f.write(f"###_Agentic_Radar_Findings_{len(result.agentic_radar_findings)}\n\n")
546             if result.agentic_radar_findings:
547                 for finding in result.agentic_radar_findings:
548                     f.write(f"_**_{finding.get('type','Unknown')}**:{finding.get('message','')} \n")
549             else:
550                 f.write("No_findings.\n")
551             f.write("\n")
552
553             # Semgrep findings
554             f.write(f"###_Semgrep_Findings_{len(result.semgrep_findings)}\n\n")
555             if result.semgrep_findings:
556                 for finding in result.semgrep_findings:
557                     severity = finding.get('severity', 'UNKNOWN')
558                     rule = finding.get('rule_id', 'unknown')
559                     message = finding.get('message', '')
560                     line = finding.get('line', 0)
561                     f.write(f"_**_{severity}**_{rule}_{line}_{line}**:_{message}\n")
562             else:
563                 f.write("No_findings.\n")
564             f.write("\n")
565             f.write("---\n\n")
566
567     print(f"[+]Markdown_report_saved_to:{output_path}")
568
569
570 # =====
571 # CLI Interface
572 # =====
573
574 def main():
575     """Main_entry_point_for_the_workflow_analyzer"""
576     parser = argparse.ArgumentParser(
577         description="n8nWorkflowSecurityAnalyzer_-_Hybrid_analysis_using_Agentic_Radar_and_Semgrep",
578         formatter_class=argparse.RawDescriptionHelpFormatter,
579         epilog="""
580 Examples:
581 _ _Analyze_single_workflow
582 _ %(prog)s_workflow_1.json
583
584 _ _Analyze_with_custom_output_directory
585 _ %(prog)s_workflow_1.json--output_results/
586
587 _ _Batch_analysis
588 _ %(prog)s--batch_./workflows/
589
590 _ _Generate_JSON_report
591 _ %(prog)s_workflow_1.json--format_json--report_output.json
592 """
593     )
594
595     # Input arguments
596     parser.add_argument(
597         'workflow',
598         nargs='?',
599         help='Path_to_workflow_JSON_file'
600     )

```



```

601 parser.add_argument(
602     '--batch',
603     metavar='DIR',
604     help='Analyze all workflows in directory'
605 )
606
607 # Output arguments
608 parser.add_argument(
609     '--output',
610     metavar='DIR',
611     default='analysis_output',
612     help='Output directory for analysis results (default: analysis_output)'
613 )
614 parser.add_argument(
615     '--rules',
616     metavar='PATH',
617     default='rules/n8n-generic-patterns.yaml',
618     help='Path to Semgrep rules file (default: rules/n8n-generic-patterns.yaml)'
619 )
620
621 # Report arguments
622 parser.add_argument(
623     '--format',
624     choices=['json', 'markdown', 'both'],
625     default='markdown',
626     help='Report format (default: markdown)'
627 )
628 parser.add_argument(
629     '--report',
630     metavar='PATH',
631     help='Path for generated report (default: auto-generated in output dir)'
632 )
633
634 args = parser.parse_args()
635
636 # Validate input
637 if not args.workflow and not args.batch:
638     parser.error("Either workflow file or --batch directory must be specified")
639
640 if args.workflow and args.batch:
641     parser.error("Cannot specify both workflow file and --batch directory")
642
643 # Initialize analyzer
644 analyzer = HybridAnalyzer(
645     agentic_output_dir=args.output,
646     semgrep_rules=args.rules
647 )
648
649 # Perform analysis
650 results = []
651 if args.batch:
652     results = analyzer.analyze_batch(args.batch)
653 else:
654     result = analyzer.analyze(args.workflow)
655     if result:
656         results = [result]
657
658 if not results:
659     print("\n[!] No successful analyses to report")
660     return 1
661
662 # Generate reports
663 print("\n" + "="*70)
664 print("ANALYSIS COMPLETE")
665 print("="*70)
666
667 timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
668
669 if args.format in ['json', 'both']:
670     json_path = args.report if args.report and args.format == 'json' else \
671         f"{args.output}/report_{timestamp}.json"
672     generate_json_report(results, json_path)
673
674 if args.format in ['markdown', 'both']:
675     md_path = args.report if args.report and args.format == 'markdown' else \
676         f"{args.output}/report_{timestamp}.md"
677     generate_markdown_report(results, md_path)
678
679 # Print summary
680 print(f"\n{'='*70}")
681 print("SUMMARY")
682 print("="*70)
683 print(f"Workflows analyzed: {len(results)}")
684 print(f"Total findings: {sum(r.total_findings_for_r in results)}")
685 print(f"Average execution time: {sum(r.execution_time_for_r in results)/len(results):.2f}s")
686
687 return 0
688
689 if __name__ == '__main__':
690     sys.exit(main())
691

```

B Regras Semgrep para n8n

Arquivo de configuração contendo as 7 regras Semgrep desenvolvidas para detecção de vulnerabilidades em workflows n8n.

Listing 6: rules/n8n-generic-patterns.yaml - Regras de segurança para n8n

```
1 rules
2   - id n8n-pgpassword-injection
3     message |
4       Detected PGPASSWORD with n8n dynamic expression. Password could contain
5       shell metacharacters leading to command injection.
6     severity ERROR
7     languages
8       - generic
9     metadata
10      category security
11      cwe CWE-78
12    patterns
13      - pattern-regex 'PGPASSWORD.*\{\{.*\}\}'
14
15   - id n8n-ssh-key-exposed
16     message |
17       Detected hardcoded SSH key. Keys should not be embedded in workflows.
18     severity WARNING
19     languages
20       - generic
21     metadata
22      category security
23      cwe CWE-798
24    patterns
25      - pattern-regex 'ssh-(rsa|ed25519|dss) \s+[A-Za-z0-9+/=]{50,}'
26
27   - id n8n-command-with-expression
28     message |
29       SSH or execute command with n8n expression. Risk of command injection.
30     severity ERROR
31     languages
32       - generic
33     metadata
34      category security
35      cwe CWE-78
36    patterns
37      - pattern-regex '"command":\s*=".*\{\{.*\$json.*\}\}'
38
39   - id n8n-hardcoded-password-in-json
40     message |
41       Hardcoded password found in workflow. Use n8n credentials instead.
42     severity ERROR
43     languages
44       - generic
45     metadata
46      category security
47      cwe CWE-798
48    patterns
49      - pattern-regex '"password":\s*" [A-Za-z0-9!@#%&*]{8,}"'
50      - pattern-not-regex '\{\{'
51
52   - id n8n-sql-injection-risk
53     message |
54       SQL query with dynamic n8n expression. Risk of SQL injection.
55     severity ERROR
56     languages
57       - generic
58     metadata
59      category security
60      cwe CWE-89
61    patterns
62      - pattern-regex '(SELECT|INSERT|UPDATE|DELETE|FROM|WHERE).*\{\{.*\$json'
63
64   - id n8n-delete-dynamic-url
65     message |
66       HTTP DELETE with dynamic URL. Risk of unauthorized deletion.
67     severity ERROR
68     languages
69       - generic
70     metadata
71      category security
72      cwe CWE-285
73    patterns
74      - pattern-regex '"method":\s*"DELETE"'
75      - pattern-regex '"url":\s*=".*\{\{.*\}\}'
76
77   - id n8n-vault-secret-operation
78     message |
79       Operation on Vault secrets endpoint. Ensure proper authorization.
80     severity WARNING
81     languages
82       - generic
83     metadata
84      category security
85    patterns
86      - pattern-regex 'vault.*/(metadata|data)/.*\{\{.*\}\}'
```